

Отчёт по лабораторной работе № 6

Название работы: Интерактивный rebase, cherry-pick, теги, reflog

Студент: Вандышев Андрей Михайлович

Группа: AT-15

Дата выполнения: 11.05.2026

1. Выполнение шагов лабораторной работы

Отметьте галочками выполненные шаги. В столбце «Команды / вывод» напишите выполненные команды и полученный вывод (поле автоматически растягивается по содержимому).

Шаг	Описание	Вып.	Команды / вывод
1	Настроить редактор Git на nano (git config --global core.editor 'nano')	☒	\$ git config --global core.editor "nano"
2	Создать репозиторий и сделать 5 коммитов (Step 1..5)	☒	\$ git init Initialized empty Git repository in C:/Users/ Пользователь/Desktop/lab6_advanced /.git/ \$ for i in {1..5}; do echo "Commit \$i" > f\$i.txt; git add .; git commit -m "Step \$i"; done \$ git log --oneline ccf3761 (HEAD -> master) Step 5 08fb272 Step 4 05be031 Step 3 551ebf2 Step 2 8d5091f Step 1
3	Создать ветку feature-rebase, добавить три коммита (Feature A, Feature B, Fix typo)	☒	\$ git checkout -b feature-rebase Switched to a new branch 'feature-rebase' \$ echo "Feature A" > fa.txt; git add .; git

Шаг	Описание	Вып.	Команды / вывод
			commit -m "Feature A" [feature-rebase b1bde16] Feature A 1 file changed, 1 insertion(+) create mode 100644 fa.txt \$ echo "Feature B" > fb.txt; git add .; git commit -m "Feature B" [feature-rebase 6b29a80] Feature B 1 file changed, 1 insertion(+) create mode 100644 fb.txt \$ echo "Fix" >> f3.txt; git commit -am "Fix typo" [feature-rebase f4eb792] Fix typo 1 file changed, 1 insertion(+) \$ git log --oneline f4eb792 (HEAD -> feature-rebase) Fix typo 6b29a80 Feature B b1bde16 Feature A ccf3761 (master) Step 5
4	Запустить git rebase -i HEAD~3 и изменить: squash для Feature B, reword для Fix typo	✓	\$ git rebase -i HEAD~3
5	Сохранить изменения в редакторе (nano: Ctrl+O, Enter, Ctrl+X)	✓	(Выполнено в редакторе nano. Изменено pick на squash для второго коммита и pick на reword для третьего. Сохранено через Ctrl+O, Enter, Ctrl+X)
6	При squash объединить сообщения коммитов, при reword написать 'Fix critical typo in f3.txt'	✓	[detached HEAD 678c7de] Feature A Date: Mon May 11 15:04:42 2026 +0500 2 files changed, 2 insertions(+) create mode 100644 fa.txt create mode 100644 fb.txt [detached HEAD 24a8c0f] Fix critical typo in f3.txt Date: Mon May 11 15:04:43 2026 +0500 1 file changed, 1 insertion(+)

Шаг	Описание	Вып.	Команды / вывод
			Successfully rebased and updated refs/heads/feature-rebase.
7	Проверить историю (git log --oneline) — коммитов стало меньше	✓	<pre>\$ git log --oneline 24a8c0f (HEAD -> feature-rebase) Fix critical typo in f3.txt 678c7de Feature A ccf3761 (master) Step 5 08fb272 Step 4 05be031 Step 3 551ebf2 Step 2 8d5091f Step 1</pre>
8	Переключиться в main и скопировать исправленный коммит через cherry-pick	✓	<pre>\$ git checkout master Switched to branch 'master' \$ git cherry-pick 24a8c0f [master d4c2caf] Fix critical typo in f3.txt Date: Mon May 11 15:04:43 2026 +0500 1 file changed, 1 insertion(+) \$ git log --oneline d4c2caf (HEAD -> master) Fix critical typo in f3.txt ccf3761 Step 5 08fb272 Step 4 05be031 Step 3 551ebf2 Step 2 8d5091f Step 1</pre>
9	Создать лёгкий тег v1.0.0 и аннотированный тег v1.1.0	✓	<pre>\$ git tag v1.0.0 \$ git tag -a v1.1.0 -m "Аннотированный тег" \$ git tag -n v1.0.0 Fix critical typo in f3.txt v1.1.0 Аннотированный тег</pre>
10	Переключиться по тегу v1.0.0 (detached HEAD), создать от него ветку hotfix-from-tag	✓	<pre>\$ git checkout v1.0.0 Note: switching to 'v1.0.0'. You are in 'detached HEAD' state. HEAD is now at d4c2caf Fix critical typo in f3.txt</pre>

Шаг	Описание	Вып.	Команды / вывод
			\$ git checkout -b hotfix-from-tag Switched to a new branch 'hotfix-from-tag'
11	Выполнить git reset --hard HEAD~3, затем восстановить потерянные коммиты через reflog	✓	\$ git checkout master Switched to branch 'master' \$ git reset --hard HEAD~3 HEAD is now at 05be031 Step 3 \$ git reflog 05be031 HEAD@{0}: reset: moving to HEAD~3 d4c2caf HEAD@{1}: checkout: moving from hotfix-from-tag to master (вывод сокращен) \$ git reset --hard HEAD@{1} HEAD is now at d4c2caf Fix critical typo in f3.txt

2. Самостоятельное задание

Условие:

Создайте ветку messy, сделайте 4 коммита. С помощью git rebase -i HEAD~4 объедините первые два коммита и переименуйте третий. Перенесите один коммит из messy в main через cherry-pick. Создайте аннотированный тег exercise-v1.

Выполнение (ход действий и результат):

```
$ git checkout -b messy
$ echo "File 1" > m1.txt; git add .; git commit -m "Commit 1"
$ echo "File 2" > m2.txt; git add .; git commit -m "Commit 2"
$ echo "File 3" > m3.txt; git add .; git commit -m "Commit 3"
$ echo "File 4" > m4.txt; git add .; git commit -m "Commit 4"
```

```
$ git rebase -i HEAD~4
```

(В редакторе nano изменено: pick на squash для Commit 2, pick на reword для Commit 3. Введено сообщение "Renamed Commit 3")

```
$ git checkout master
Switched to branch 'master'
```

```
$ git cherry-pick a1f5548
[master b953182] Renamed Commit 3
```

Date: Mon May 11 15:18:29 2026 +0500

1 file changed, 1 insertion(+)

create mode 100644 m3.txt

```
$ git tag -a exercise-v1 -m "Exercise tag"
```

```
$ git log --oneline
```

```
b953182 (HEAD -> master, tag: exercise-v1) Renamed Commit 3
```

```
d4c2caf (tag: v1.1.0, tag: v1.0.0, hotfix-from-tag) Fix critical typo in f3.txt
```

```
ccf3761 Step 5
```

```
08fb272 Step 4
```

```
05be031 Step 3
```

```
551ebf2 Step 2
```

```
8d5091f Step 1
```

Результат:

Ветка messy успешно создана. История 4-х коммитов переписана с помощью интерактивного rebase: первые два коммита объединены (squash), а третий переименован (reword). Выбранный коммит (Renamed Commit 3) успешно скопирован в ветку master с помощью cherry-pick, и на него установлен аннотированный тег exercise-v1.

3. Ответы на контрольные вопросы

1. Вопрос 1:

Rebase полностью переписывает историю Git, создавая новые коммиты с новыми хэшами. Если применить его к ветке, которая уже отправлена на удалённый сервер, и другие разработчики уже скачали её себе, это приведет к рассинхронизации историй. При попытке объединить работу возникнут серьезные конфликты и путаница.

2. Вопрос 2:

Команда `git merge` объединяет истории целых веток вместе, создавая коммит слияния. В свою очередь `git cherry-pick` берет изменения только из одного конкретного коммита любой ветки и применяет их к текущей ветке как абсолютно новый коммит, не связывая истории веток между собой.

3. Вопрос 3:

Нужно использовать команду `git reflog`. Она покажет локальный журнал всех перемещений указателя HEAD. В этом журнале нужно найти хэш последнего правильного коммита (до выполнения ошибочного сброса, например `HEAD@{1}`) и выполнить команду `git reset --hard <найденный_хэш>`.

4. Дополнительные замечания (при необходимости)

Тут нет отдельного раздела для ответа на вопрос 4: "Как сменить редактор по умолчанию для Git?", так что я отвечу здесь. Редактор по умолчанию меняется через глобальные настройки конфигурации Git. Для этого нужно выполнить команду `git config --global core.editor "<название_редактора>"`. Например, для установки простого консольного редактора nano используется команда `git config --global core.editor "nano"`.

Инструкция: Выберите номер лабораторной работы. Заполняйте поля — они автоматически растягиваются по высоте под текст. Для печати нажмите кнопку «Печать» — все рамки полей исчезнут, и отчёт будет выглядеть как обычный документ.